Memory-Mapped I/O

- **Memory-Mapped I/O**: Refers to communicating with a hardware device by mapping a area of memory to represent the device. Instead of using special I/O instructions, the system can read and write to specific memory addresses that correspond to device registers, simplifying operations.
    - **Example**: In UNIX, everything (files, device) is treated like a file, similarly here, devices are treated like memory

Port-Mapped vs. Memory-Mapped I/O

- **Port-Mapped I/O**: The older method where special CPU instructions and registers were used to communicate with devices. This method is more difficult to work with and less efficient compared to memory-mapped I/O
- **Memory-Mapped Control**: By mapping device control registers to memory addresses, we can control devices by reading and writing values (1s and 0s) directly to these addresses, just like accessing memory

Peripheral Devices and Registers

- **Peripheral Devices**: Hardware components like UART, I2C, GPIO are used outside the CPU but communicate with it. These protocols interact with the CPU through memory-mapped I/O.
    - **Pins**: The physical connectors that link the device to the motherboard are called **pins**. Each pin can have different function but only one function at a time
    - **Control via Registers**: Devices are often controlled by writing to specific registers. Registers are divided into:
        a. **Configuration Registers**: Set up the device
        b. **Data Registers**: Store the data being read or written
        c. **Control Registers**: Instruct the device on what to do

STM32 Chip Example

- **Base + Offset Approach**: On STM32 chips, devices like GPIO are controlled using a **base address** with specific **offsets** for different registers. This allows support for multiple devices
    - **Example**: The base address for GPIO might be provided in the memory map, and the specific register offset for controlling the device is in the GPIO documentation
    - **Global Enable (RCC)**: Some peripherals require a global enable signal (like a clock) to function. Without this signal, the peripheral will remain inactive to save power

GPIO (General-Purpose Input/Output)

- **GPIO**: Controls the volatge levels on specific pins, which are used for simple operations like turning LEDs on/off or detecting signals
    - **Usage**:
        - **Input**: Detecting signals (e.g. button presses)
        - **Output**: Controlling devices (e.g. turning on LEDs)
        - **Analog Input**: Reading sensor data
        - **Alternate Functions**: Some pins can serve special purposes defined by the chip manufacturer
- **GPIO Registers**:
    i. **GPIOx_MODER**: Determines the mode (input/output/analog/alternate function)
    ii. **GPIOx_OTYPER**: Sets the output type (push-pull or open-drain)
    iii. **GPIOx_OSPEEDR**: Sets the data transfer speed
    iv. **GPIOx_PUPDR**: Enables pull-up or pull-down resistors for input pins

Basic GPIO Example

- **Example Setup**:
    - **GPIO Input (Reading a Button)**:

```
void setGPIOInput() {
  _GPIO_CLOCK_EN |= 1 << 2;
  _GPIO_MODDER &= ~(0x3 << 26); // Set PC13 as input
}
```

- **GPIO Output (Controlling an LED)**:

```
void setGPIOAOutput() {
  _GPIOA_MODDER &= ~(0x3 << 10); // Clear PA5
  _GPIOA_MODDER |= 1 << 10; // Set PA5 as output
}
```

- **Accessing GPIO**:
  - **Reading a Pin (Input)**:

```
int readGPIOC() {
    return (_GPIOC_IDR & (1 << 13)) >> 13; // Read PC13
}
```

  - **Writing to a Pin (Output)**:

```
void writeGPIOA(bool in) {
    if (in) {
        _GPIOA_ODR |= (1 << 15); // Set PA5 HIGH
    } else {
        _GPIOA_ODR &= ~(1 << 5); // Set PA5 LOW
    }
}
```

UNIX mmap() for Memory Mapping Files

- In UNIX based systems, the `mmap()` function allows mapping a file into memory so it can be cassesed like a block of memory rather than through the file I/O operations
  - `mmap()` **Function Parameters**:
    a. **Address**: Where in memory the file should be mapped (NULL to let the OS choose)

    b. **Length**: Number of bytes to map
    c. **Protection**: Access rules (e.g. read, write, private)
    d. **Flag**: Mapping operations (e.g., private or shared)
    e. **Offset**: Where to start mapping within the file
- **Protection Flags**: Specify read (`PROT_READ`), write (`PROT_WRITE`), execute (`PROT_EXECUTE`), or none (`PROT_NONE`)
- **Example**: Mapping file into memory, modifying it, and writing the changes back:

```
int fd = open("example.txt", O_RDWR);
struct stat st;
stat("example.txt", &st)
ssize_t size = st.st_size;
void *mapped = mmap(NULL, size, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);

int pid = fork();
if (pid > 0) { // Parent process
    waitpid(pid, NULL, 0);
    printf("The new content of the file is: %s.\n", (char *)mapped);
    munmap(mapped, size);
} else if { // Child process
    memset(mapped, 0, size);
    sprintf(mapped, "It is now Overwritten");
    msync(mapped, size, MS_SYNC);
    munmap(mapped, size);
}
close(fd)
```

Conclusion

Memory-mapped I/O simplifies interactions with hardware devices by treating device registers as memory addresses. This allows for easier, more efficient control over peripherals like GPIO and communication buses. Additionally, memory mapping techniques in operating systems like UNIX (mmap()) enable efficient file manipulation by mapping files directly into memory. This combination of memory-mapped I/O and file mapping is essential for modern embedded systems and performance-critical applications.